

低剂量 CT 图像的传统去噪 与插值增强方法对比研究

李建辉 苏威贤 王妍 李定一

摘要 针对低剂量 CT (Low-Dose CT, LDCT) 图像噪声较强、细节退化等问题, 本文采用传统数字图像处理方法进行去噪与超分辨率重建。实验比较均值、高斯、自适应中值、非局部均值 (Non-Local Means, NLM) 和小波软阈值五种去噪方法, 定量分析结果表明, 在本实验设置下, NLM 综合表现最佳, 峰值信噪比 (PSNR) 和结构相似性 (SSIM) 分别为 39.252 dB 和 0.95269, 较原始 LDCT 分别提高 2.368 dB 和 0.04353。进一步以 NLM 去噪结果构造低分辨率输入, 比较最近邻、双线性和双三次三种 4 倍插值方法, 结果表明双三次插值取得最高 PSNR 28.005 dB 和 SSIM 0.88673。顺序实验表明, 先去噪后超分优于先超分后去噪。综合实验结果, 本文推荐采用“先 NLM 去噪、后双三次插值”的处理流程。

关键词 低剂量 CT; 图像去噪; 非局部均值; 超分辨率重建; 双三次插值; 结构相似性

Abstract To address the problems of strong noise and detail degradation in low-dose computed tomography (LDCT) images, this study employs traditional digital image processing methods for denoising and super-resolution reconstruction. Five denoising methods, including mean filtering, Gaussian filtering, adaptive median filtering, Non-Local Means (NLM), and wavelet soft-thresholding, are compared. Quantitative results show that, under the experimental settings of this study, NLM achieves the best overall performance, with a peak signal-to-noise ratio (PSNR) of 39.252 dB and a structural similarity index (SSIM) of 0.95269, representing improvements of 2.368 dB and 0.04353, respectively, over the original LDCT image. Furthermore, low-resolution inputs are constructed from the NLM-denoised results, and three 4× interpolation methods, namely nearest-neighbor, bilinear, and bicubic interpolation, are compared. The results show that bicubic interpolation achieves the highest PSNR of 28.005 dB and SSIM of 0.88673. The processing-order experiment further shows that denoising before super-resolution performs better than super-resolution before denoising. Based on the overall experimental results, the processing pipeline of “NLM denoising followed by bicubic interpolation” is recommended under the experimental conditions of this study.

Keywords low-dose computed tomography; image denoising; Non-Local Means; super-resolution reconstruction; bicubic interpolation; structural similarity

1 引言

低剂量 CT 通过降低管电流或缩短曝光时间减少辐射剂量，但光子数不足会引入强烈的泊松噪声、条状伪影和颗粒感^[1]，同时受成像系统空间分辨率限制，肺小结节、细小支气管和末梢血管的边缘容易模糊。但去噪与细节保留之间存在天然矛盾：平滑滤波能抑制噪声，却会模糊边缘和细小结构；保边滤波能保留结构，却可能残留噪声或产生伪影。传统插值超分也只能基于已有像素估计新像素，无法恢复采集或降采样过程中已经丢失的真实高频信息^[2-3]。我们的目标是在完全不使用神经网络的前提下，利用传统图像处理方法对低剂量 CT 进行噪声抑制和 4 倍超分辨率增强，并以严格配对的全剂量 CT 为参考，评价不同去噪与插值组合的效果。我们读取配对 DICOM 切片并进行肺窗预处理；比较均值滤波、高斯滤波、自适应中值滤波、非局部均值滤波和小波软阈值去噪；选取代表性去噪结果进行最近邻、双线性插值和双三次插值放大；以全剂量 CT 为真值，结合主观视觉和 PSNR、SSIM 指标评价重建质量，并分析去噪与超分的先后顺序对最终结果的影响。

2 相关工作

2.1 传统空间域去噪

空间域去噪直接对像素邻域进行运算。均值滤波以邻域均值替代中心像素，计算简单，但会模糊边缘和细小结构。高斯滤波使用高斯核加权，平滑效果比均值滤波自然，但核尺寸和标准差增大时仍会损失细节^[2]。中值滤波对脉冲噪声有较好抑制作用，但普通中值滤波对 CT 颗粒噪声和低对比结构保持能力有限。自适应中值滤波根据局部窗口内灰度分布动态调整窗口大小，在抑制噪声的同时尽量保护边缘^[4]。非局部均值滤波利用图像中重复纹理和自相似结构，通过图像块相似度加权平均来估计像素，利用非局部自相似信息兼顾噪声抑制与结构保持，但计算量较大，滤波参数对结果影响明显^[5]。总体而言，空间域方法实现简单、易于控制，但线性平滑容易模糊细节，非线性方法在噪声抑制和结构保持之间各有取舍。

2.2 小波域去噪

小波变换将图像分解为低频近似和不同方向、不同尺度的高频细节。低剂量 CT 随机噪声多集中在高频子带，因此可对高频系数进行阈值收缩。软阈值函数为^[6]

$$T_{\lambda}(x) = \text{sgn}(x) \max(|x| - \lambda, 0)$$

其中 λ 为阈值。阈值过大时会丢失小结节和血管边缘，阈值过小时噪声残留明显。与小波相比，空间域方法直接在像素上操作，计算更简单，小波域方法提供多尺度表示，但噪声和真实结构均可能包含高频成分，其效果取决于阈值和图像特征，不能仅凭变换域表示保证医学图像细节得到更好保留。本实验选择小波软阈值作为变换域去噪的代表方法。

2.3 图像插值增强

传统超分辨率插值根据低分辨率图像中已有像素的灰度连续性估计高分辨率网格上的像素值^[2]。最近邻插值将目标像素灰度赋值为距离最近的源像素灰度，计算简单、速度快，但放大后在斜线和曲线边缘容易产生明显锯齿和块状效应。双线性插值利用目标点周围 2×2 邻域像素进行加权平均，比最近邻插值连续，能削弱块状效应，但加权平均会平滑高频成分，使细小血管和支气管边缘变软、变模糊。双三次插值利用周围 4×4 像素，通过三次多项式核进行加权，在边缘连续性、细节保持和计算复杂度之间取得较好折中，具

体结构相似性需由实验判断, 在强边缘附近也可能出现过冲或振铃^[3]。三种方法都只能改善显示分辨率和视觉连续性, 不能恢复降采样或低剂量采集过程中已经丢失的真实高频医学信息。

3 实验方法与处理流程

3.1 总体流程

本实验使用 AAPM 2016 Mayo LDCT-and-Projection-data 中的配对数据^[7], 同一患者、同一扫描位置的低剂量 CT 与全剂量 CT 图像严格配准。低剂量 CT 作为低质量输入, 全剂量 CT 作为高质量参考真值。总体流程为: 读取配对 DICOM 切片并进行肺窗预处理; 对低剂量 CT 分别执行五种传统去噪; 以结构相似性优先、峰值信噪比决胜的规则选取代表性去噪结果; 将去噪图像降采样后分别用三种插值方法放大 4 倍; 最后以全剂量 CT 为参考, 进行主观可视化和客观指标评价, 并设计先降噪后超分与先超分后降噪两条流程进行顺序对照。

3.2 DICOM 读取、切片配对、HU 截断与归一化

使用 SimpleITK 读取 DICOM 格式的成对低剂量 CT 和全剂量 CT 切片, 检查患者编号、图像位置、图像方向、像素间距、层厚、重建核和实例编号等元数据, 确认两者在几何上对应。DICOM CT 的像素到 HU 转换遵循 Rescale Slope 和 Rescale Intercept 定义; 本实验由 SimpleITK 读取图像后直接使用其数值数组, 避免重复施加重标定^[8]。然后进行 HU 值截断并设置肺窗, 窗位为 -600 HU, 窗宽为 1500 HU, 对应 HU 范围从 -1350 到 150 HU。将截断后的 HU 值线性映射为 uint8 灰度图:

$$I_{\text{uint8}} = \text{clip} \left(\frac{\text{HU} + 1350}{1500} \times 255, 0, 255 \right) \quad (1)$$

对低剂量 CT 和全剂量 CT 采用完全相同的窗宽窗位和归一化方式, 选取 L067 的一组成对切片作为后续去噪、插值和评价的输入; 本实验未获得肺结节标注, 不把局部结构的视觉对比视为结节识别验证。

3.3 均值、高斯、自适应中值、NLM、小波软阈值五种去噪方法

对原始低剂量 CT 分别执行五种去噪。均值滤波改变滤波核大小; 高斯滤波改变滤波核大小和标准差; 自适应中值滤波改变最大窗口尺寸; 非局部均值滤波改变滤波参数 h , 其估计为^[5]

$$\hat{I}(i) = \frac{1}{Z(i)} \sum_{j \in \Omega} w(i, j) I(j), \quad w(i, j) = \exp \left(-\frac{\|v(i) - v(j)\|_2^2}{h^2} \right) \quad (2)$$

其中 $Z(i)$ 为归一化因子; 小波软阈值去噪固定小波基为 db1、分解层数为 2, 仅搜索阈值^[6]。每种方法均在预设参数网格上搜索, 参数选择规则为优先最大化结构相似性, 结构相似性相同时以峰值信噪比作为决胜项。该规则体现医学图像对结构保持的重视, 具体参数搜索结果和最终指标将在后续实验设置与结果分析部分给出。参数搜索与选优实现见附录代码 `task2/denoise_dicom.py`。

3.4 最近邻、双线性、双三次三种 4 倍插值方法

读取去噪图像, 使用区域插值将图像缩小到低分辨率, 模拟低分辨率输入, 然后分别采用最近邻插值、双线性插值和双三次插值将低分辨率图像放大 4 倍恢复原尺寸。最近邻插值直接复制最近像素; 双线性插值利用周围 2×2 邻域加权; 双三次插值利用周围 4×4 像素通过三次多项式核加权。保证三种结果尺寸和

灰度范围一致，重点观察组织轮廓、局部纹理和高对比边缘，并以降采样前的去噪图像作为本任务内部参考，比较三种插值方法对已有结构的近似能力。

3.5 PSNR、SSIM 指标与计算条件

客观评价以配对全剂量 CT 高质量图像作为参考真值，计算峰值信噪比和结构相似性。峰值信噪比为^[2]

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right) \quad (3)$$

其中 MAX_I 为图像最大灰度值，本实验归一化到 uint8 后取 255；MSE 为均方误差。结构相似性为^[9]

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (4)$$

评价范围为完整图像，不包含标题、坐标轴和白边，避免非图像区域影响指标。需要注意，任务三的插值内部对比以降采样前的去噪图像为参考，任务四的最终超分评价以全剂量 CT 原始图为参考，两者参考对象不同，指标数值不能直接混用。此外，顺序影响实验设计先降噪后超分和先超分后降噪两条流程，本报告任务五的现行流程直接将 512×512 的图像插值放大至 2048×2048 ，参考图像为采用同样双三次插值放大的 FDCT。该参考是尺寸对齐的插值图像，不是实际采集的高分辨率真值，不能与任务三、四的指标直接混用；顺序实验实现见附录代码 `task5/task5-1.py`。

4 实验设置与结果分析

4.1 实验设置

实验数据来源于 AAPM2016 Mayo LDCT 数据集，选取病人 L067 的低剂量 CT 和全剂量 CT 切片各一张。两张图像来自患者的同一部位，图像尺寸为 512×512 ，像素间距 $0.6640625 \times 0.6640625$ mm，层厚 3 mm，重建核 B30f。

预处理采用肺窗，窗位 -600 HU，窗宽 1500 HU，HU 范围 $[-1350, 150]$ ，线性归一化到 uint8 0 ~ 255。

实验使用 Windows 和 Linux 为运行平台，主要依赖库包括 NumPy、OpenCV、SimpleITK、PyWavelets、scikit-image 和 Matplotlib。

实验的评价指标为 PSNR 和 SSIM。其中 PSNR 为像素误差，单位为 dB。SSIM 为结构相似性，越接近 1 则越好。实验以全剂量 CT 为 GT 图像，`data_range` 设置为 255。评价时，以 SSIM 为优先指标，SSIM 相同时则比较 PSNR。注意本实验的参数搜索与最终评价均在同一测试切片上完成，实验结果主要用于方法间的相对比较，不用于证明模型在未见数据上的泛化能力。

最终去噪参数为：均值核 3×3 ；高斯核 3×3 、 $\sigma = 1.0$ ；自适应中值最大窗口 5；NLM 的 $h = 4$ 、模板窗口 7、搜索窗口 21；小波基 db1、分解 2 层、阈值 4。具体实现见附录代码 `task2/denoise_dicom.py`。

超分实验部分以 NLM 去噪结果为输入，先使用 `INTER_AREA` 降采样到 128×128 ，再分别使用最近邻、双线性、双三次插值放大 4 倍恢复为 512×512 。

4.2 去噪结果

如表 1 所示，在五种去噪结果的对比中，NLM 的指标表现最优，其 $\text{SSIM}=0.95269$ ， $\text{PSNR}=39.252$ dB。

Table 1 五种去噪方法最优指标 (以 FDCT 为 GT)

方法	PSNR (dB)	SSIM	Δ PSNR (dB)	Δ SSIM
LDCT (原始)	36.884	0.90915	0.000	0.00000
均值滤波	35.105	0.94158	-1.778	+0.03243
高斯滤波	36.120	0.94220	-0.763	+0.03305
自适应中值滤波	36.768	0.92422	-0.115	+0.01507
NLM	39.252	0.95269	+2.368	+0.04353
小波软阈值	37.668	0.93691	+0.785	+0.02775

如表 1 所示, NLM 的 PSNR 为 39.252 dB, SSIM 为 0.95269, 在五种方法中综合指标最优。相比原始 LDCT, NLM 的 PSNR 提高 2.368 dB, SSIM 提高 0.04353。

均值滤波的 SSIM 提高到 0.94158, 但 PSNR 下降至 35.105 dB, 说明滤波在一定程度上改善了图像结构相似性, 但可能导致部分细节损失。高斯滤波和自适应中值滤波的 PSNR 也低于原始 LDCT, 同样存在细节损失。小波软阈值的 PSNR 为 37.668 dB, SSIM 为 0.93691, 优于均值、高斯和自适应中值, 但仍低于 NLM。

综合来看, NLM 在抑制噪声和保留局部纹理与组织边缘方面表现最好, 因此后续超分任务选用 NLM 去噪结果作为输入。图 1 给出原始 LDCT、五种去噪结果与 FDCT 的并排对比。

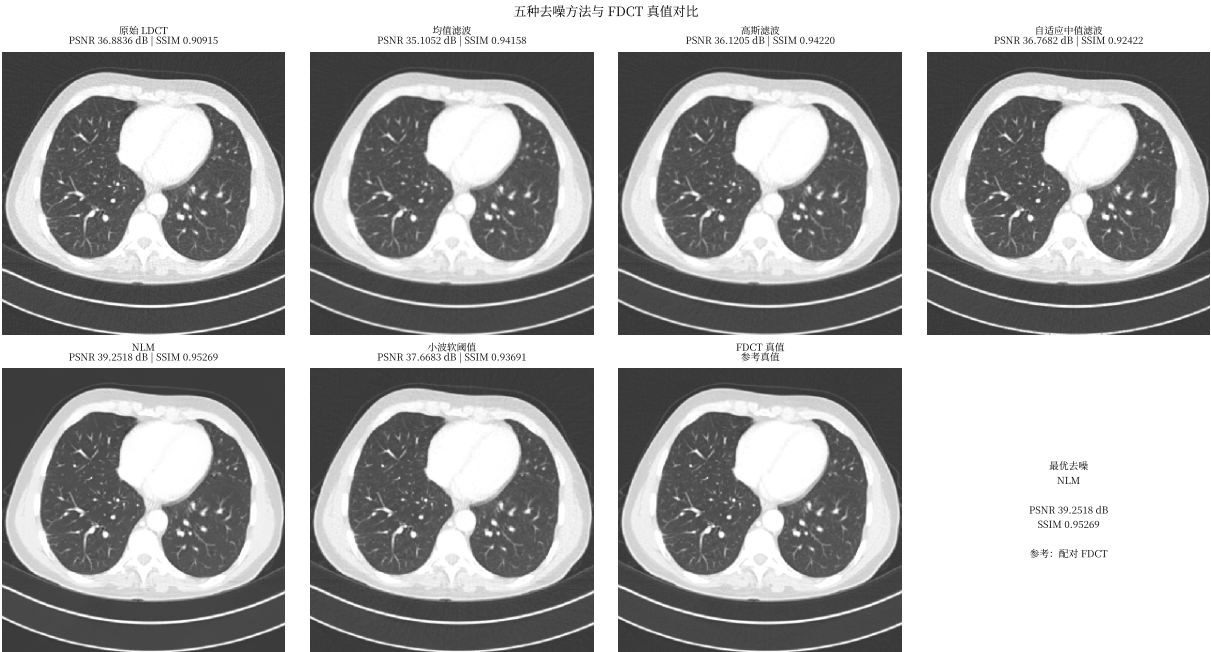


Fig. 1 原始 LDCT、五种去噪结果与 FDCT 对比

4.3 超分辨率重建结果

如表 2 所示, 在 NLM 去噪的基础上, 三种插值方式中双三次插值的结果最优。

Table 2 三种插值方法重建指标（参考图像为降采样前的 NLM 图像）

方法	PSNR (dB)	SSIM	梯度边缘 MAE	梯度相关系数
最近邻	24.665	0.84879	33.539	0.6647
双线性	26.270	0.86476	27.400	0.8140
双三次	28.005	0.88673	25.157	0.8437

如表 2 所示，最近邻插值的 PSNR、SSIM 和梯度相关系数在三种方法中均为最低，边缘误差也最大，说明其重建效果较差。双线性插值能够明显减少最近邻插值产生的锯齿，但局部细小结构边缘仍较为模糊。相比之下，双三次插值的 PSNR (28.005 dB) 和 SSIM (0.88673) 均最高，梯度边缘 MAE 最低 (25.157)，梯度相关系数最高 (0.8437)，其重建边缘与降采样前的 NLM 参考图像最接近。

三种方法的综合效果排序为：双三次插值 > 双线性插值 > 最近邻插值。双三次插值在边缘连续性、细节保持和定量指标之间取得了最好的平衡，在本实验设置下表现最佳。

需要注意，双三次插值仍然只是根据已有像素进行估计。它可以改善显示的平滑性和视觉连续性，但不能恢复在 128×128 降采样过程中丢失的真实高频医学信息。图 2 给出 NLM、三种插值重建与 FDCT 的并排对比。



Fig. 2 NLM、三种插值重建与 FDCT 对比

4.4 降噪与超分的顺序影响

为分析降噪与超分的先后顺序是否影响最终效果，设计两组方案：

- 方案 A：先降噪，后超分；
- 方案 B：先超分，后降噪。

两种流程使用数值相同的 NLM 参数 ($h = 4$ ，模板窗口 7，搜索窗口 21) 和双三次插值，仅改变执行顺序；由于处理尺寸不同，相同像素窗口对应的空间范围仍有差异，结果仅代表本次参数设置。参考图像为双三次插值放大到 2048×2048 的 FDCT，结果如表 3。

如表 3 所示，先降噪再超分得到的结果优于先超分后降噪，其 PSNR 提高了 0.77 dB，SSIM 提高了 0.0054。推测是由于降噪在原始分辨率上进行时，噪声能在放大之前被抑制。若先超分，噪声会被插值放大到更大尺度，再去噪时容易把噪声与真实边缘一起平滑，导致细节损失。因此，先降噪后超分在本实验中结果更优。

Table 3 降噪与超分顺序影响对比

方案	PSNR (dB)	SSIM
A 先降噪后超分	39.22	0.9693
B 先超分后降噪	38.45	0.9639

4.5 传统插值超分的固有局限

尽管双三次插值在三种方法中综合最优, 但从实验结果来看, 传统插值超分存在一定局限:

- (1) 无法生成真实高频细节。传统插值主要利用已有像素估计放大后的像素值, 能够改善图像的平滑性和连续性, 但不能恢复原始图像中已丢失的信息。
- (2) 边缘细节容易受到平滑影响。插值主要根据邻域像素进行估计, 对于组织轮廓等局部边缘结构, 放大过程中可能出现一定程度的平滑。
- (3) 超分后的指标提升有限, 且受到 GT 构造方式的影响。LDCT 原始 PSNR 为 36.884 dB, NLM 去噪后为 39.252 dB。而在以 FDCT 为 GT 的超分评价中, NLM+ 双三次超分后 PSNR 为 27.901 dB, 低于 NLM 去噪本身。这说明在当前实验设置下, 插值并未带来真实细节恢复, 反而使重建结果与 FDCT 之间的差异增大。
- (4) 不能替代迭代重建或学习方法。传统插值属于后处理, 无法恢复投影域丢失的信息。后续可在相同数据与评价条件下比较迭代重建等方法, 并扩展至更多患者和切片, 以检验结论的适用范围^[1,7]。

4.6 综合方案选择

综合以上结果:

- 去噪方法中, NLM 最优 (PSNR=39.252 dB, SSIM=0.95269);
- 插值方法中, 双三次最优;
- 顺序上, 先降噪后超分优于先超分后降噪。

综合以上实验结果, “先 NLM 去噪、后双三次插值”为本实验条件下的推荐处理流程。需要注意的是, 在以 FDCT 为参考的最终评价中, 双三次插值并未在客观指标上带来明显的进一步提升, 因此该流程的优势主要体现在噪声抑制以及图像尺寸和视觉连续性的改善, 而非高频医学细节的恢复。图 3 给出 LDCT、NLM、NLM+ 双三次与 FDCT 的并排对比。

5 结论

本文基于 AAPM2016 Mayo LDCT 数据集中的 L067 配对切片, 使用纯传统图像处理方法完成了低剂量 CT 的去噪与超分辨率重建, 并以全剂量 FDCT 为 Ground Truth 进行定性与定量评价。

主要结论如下:

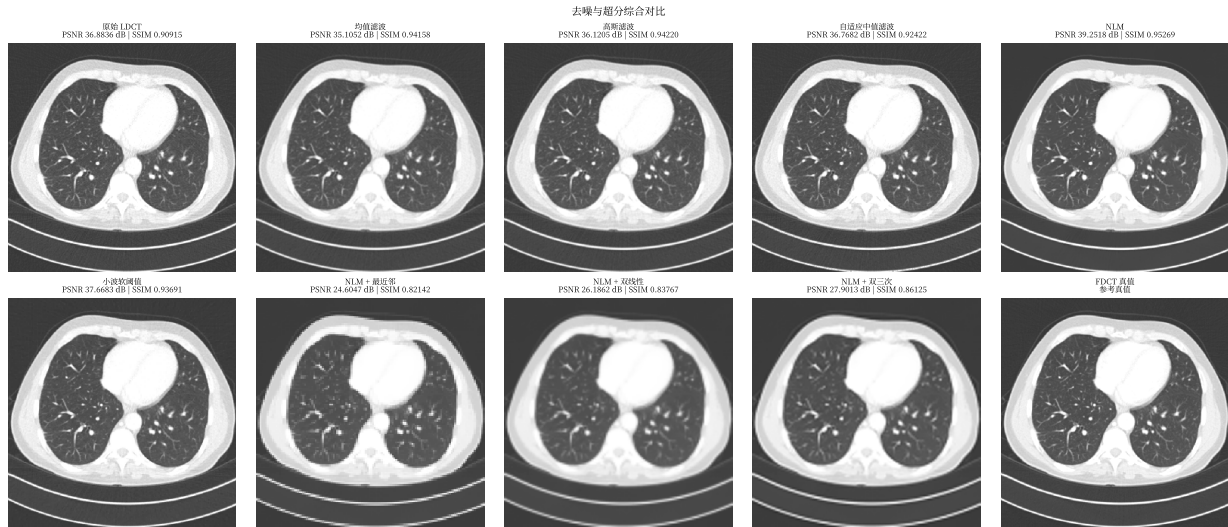


Fig. 3 LDCT、NLM、NLM+ 双三次与 FDCT 对比

- (1) 去噪方面：在均值滤波、高斯滤波、自适应中值滤波、NLM 和小波软阈值五种方法中，NLM 综合指标最优，其 PSNR 为 39.252 dB，SSIM 为 0.95269。与原始 LDCT 相比，PSNR 提高 2.368 dB，SSIM 提高 0.04353。
- (2) 超分方面：在最近邻、双线性和双三次三种插值中，双三次插值综合效果最好。在以降采样前 NLM 图像为参考的 4 倍插值实验中，双三次插值取得最高 PSNR (28.005 dB) 和 SSIM (0.88673)。
- (3) 顺序方面：先降噪后超分优于先超分后降噪。方案 A（先降噪后超分）的 PSNR 为 39.22 dB，SSIM 为 0.9693。方案 B（先超分后降噪）的 PSNR 为 38.45 dB，SSIM 为 0.9639。方案 A 比方案 B 的 PSNR 高 0.77 dB，SSIM 高 0.0054。
- (4) 推荐方案：实验结果得出，采用 NLM 进行去噪、再使用双三次插值进行 4 倍重建的方案应为本文实验的最终推荐方案。
- (5) 固有局限：传统插值超分只能改善显示的平滑性和视觉连续性，无法生成真实高频医学信息。双三次插值同样存在此局限。
- (6) 展望：后续可扩展测试患者和切片，并在相同评价条件下比较其他重建与去噪方法。本文实验限定采用传统图像处理方法。

本文实验仅基于 L067 的一张严格配对切片，结果可以验证代码和比较该切片上的算法，不能直接代表全部患者，也未把该切片宣称为已由医生确认的肺结节切片。

参考文献

- [1] YU L, LIU X, LENG S, et al. Radiation dose reduction in computed tomography: techniques and future perspective [J/OL]. *Imaging Medicine*, 2009, 1(1): 65-84. <https://doi.org/10.2217/iim.09.5>.
- [2] GONZALEZ R C, WOODS R E. Digital image processing, global edition[M/OL]. 4th ed. Pearson, 2018. <https://www.pearson.com/en-gb/subject-catalog/p/Gonzalez-Digital-Image-Processing-e-Book-Global-Edition-4th-Edition/P200000004313?view=educator>.
- [3] KEYS R G. Cubic convolution interpolation for digital image processing[J/OL]. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1981, 29(6): 1153-1160. <https://doi.org/10.1109/TASSP.1981.1163711>.
- [4] HWANG H, HADDAD R A. Adaptive median filters: new algorithms and results[J/OL]. *IEEE Transactions on Image Processing*, 1995, 4(4): 499-502. <https://doi.org/10.1109/83.370679>.
- [5] BUADES A, COLL B, MOREL J M. A non-local algorithm for image denoising[C/OL]//2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05): Vol. 2. IEEE, 2005: 60-65. <https://doi.org/10.1109/CVPR.2005.38>.
- [6] DONOHO D L. De-noising by soft-thresholding[J/OL]. *IEEE Transactions on Information Theory*, 1995, 41(3): 613-627. <https://doi.org/10.1109/18.382009>.
- [7] MCCOLLOUGH C H, BARTLEY A C, CARTER R E, et al. Low-dose ct for the detection and classification of metastatic liver lesions: Results of the 2016 low dose ct grand challenge[J/OL]. *Medical Physics*, 2017, 44(10): e339-e352. <https://doi.org/10.1002/mp.12345>.
- [8] National Electrical Manufacturers Association. Dicom ps3.3 2026d: Information object definitions, section c.8.2 ct modules[S/OL]. National Electrical Manufacturers Association, 2026[2026-10-08]. https://dicom.nema.org/medical/dicom/current/output/chtml/part03/sect_C.8.2.html.
- [9] WANG Z, BOVIK A C, SHEIKH H R, et al. Image quality assessment: from error visibility to structural similarity [J/OL]. *IEEE Transactions on Image Processing*, 2004, 13(4): 600-612. <https://doi.org/10.1109/TIP.2003.819861>.

附录实验代码

任务一: task1/main.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import SimpleITK as sitk
4 import pathlib as path
5
6 basic_path = path.Path("/home/sanxi/DIP/experiment1")
7 patient_ID="L067"
8
9 full=basic_path / "full_3mm" / patient_ID / "full_3mm" / "L067_FD_3_1.CT.0002.0001.2015.12.22.18_
    .12.07.5968.358090401.IMA"
10 quarter=basic_path / "quarter_3mm" / patient_ID / "quarter_3mm" / "L067_QD_3_1.CT.0004.0001.2015_
    .12.22.18.12.56.428910.358293547.IMA"
11 # 取出图像的中间切片
12 output_dir=basic_path / "output"
13 output_dir.mkdir(parents=True, exist_ok=True)
14
15 ct_image=sitk.ReadImage(str(full))
16 print(ct_image.GetSize())
17 ct_array=sitk.GetArrayFromImage(ct_image)
18 print(ct_array.shape)
19 ct_slice=ct_array[0]
20 print(ct_slice.shape)
21 # 将中间切片保存为 png 格式
22 plt.imshow(
23     ct_slice,
24     cmap="gray",
25     vmin=-1350,
26     vmax=150,
27     interpolation="nearest"
28 )
29 plt.title("CT Slice")
30 plt.axis('off')
31 save_path=output_dir / "ct_slice_full.png"# 保存
32 plt.savefig(save_path, dpi=300, bbox_inches='tight', pad_inches=0)
33 plt.show()
34
35
```

任务二: task2/denoise_dicom.py

```
1 #!/usr/bin/env python3
2 """ 任务二: 直接基于配对 DICOM 的传统 CT 去噪与参数比较。"""
3
4 from __future__ import annotations
5
6 import csv
7 import hashlib
8 import json
9 import platform
10 import sys
11 import time
12 from pathlib import Path
13
14 import cv2
15 import matplotlib
16
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import numpy as np
20 import pywt
21 import SimpleITK as sitk
22 import skimage
23 from skimage.metrics import peak_signal_noise_ratio, structural_similarity
24
25
26 TASK_DIR = Path(__file__).resolve().parent
27 EXPERIMENT_DIR = TASK_DIR.parent
28 RESULTS_DIR = TASK_DIR / "results"
29 FIGURES_DIR = TASK_DIR / "figures"
30 IMAGES_DIR = TASK_DIR / "images"
31
32 FDCT_PATH = (
33     EXPERIMENT_DIR
34     / "full_3mm/L067/full_3mm"
35     / "L067_FD_3_1.CT.0002.0001.2015.12.22.18.12.07.5968.358090401.IMA"
36 )
37 LDCT_PATH = (
38     EXPERIMENT_DIR
39     / "quarter_3mm/L067/quarter_3mm"
40     / "L067_QD_3_1.CT.0004.0001.2015.12.22.18.12.56.428910.358293547.IMA"
41 )
42
43 # DICOM 中提供的肺窗预设: 窗位 -600 HU, 窗宽 1500 HU。
44 WINDOW_CENTER = -600.0
45 WINDOW_WIDTH = 1500.0
46 WINDOW_MIN = WINDOW_CENTER - WINDOW_WIDTH / 2.0
47 WINDOW_MAX = WINDOW_CENTER + WINDOW_WIDTH / 2.0
48
49
50 def sha256(path: Path) -> str:
51     digest = hashlib.sha256()
52     with path.open("rb") as file:
53         for block in iter(lambda: file.read(1024 * 1024), b''):
54             digest.update(block)
55     return digest.hexdigest()
56
57
58 def read_dicom_hu(path: Path) -> tuple[np.ndarray, sitk.Image]:
59     """ 读取单张 DICOM, 并返回二维 HU 数组。"""
60     if not path.is_file():
61         raise FileNotFoundError(f" 找不到 DICOM 文件: {path}")
62     image = sitk.ReadImage(str(path))
63     array = np.squeeze(sitk.GetArrayFromImage(image)).astype(np.float32)
64     if array.ndim != 2:
65         raise ValueError(f" 期望二维切片, 实际数组形状为 {array.shape}")
66     return array, image
67
```

```

68
69 def metadata(image: sitk.Image, key: str, default: str = "N/A") -> str:
70     return image.GetMetaData(key).strip() if image.HasMetaDataKey(key) else default
71
72
73 def window_to_uint8(image_hu: np.ndarray) -> np.ndarray:
74     """ 使用固定肺窗截断, 并线性归一化为 0~255 灰度。"""
75     clipped = np.clip(image_hu, WINDOW_MIN, WINDOW_MAX)
76     normalized = (clipped - WINDOW_MIN) / (WINDOW_MAX - WINDOW_MIN)
77     return np rint(normalized * 255.0).astype(np.uint8)
78
79
80 def evaluate(reference: np.ndarray, candidate: np.ndarray) -> tuple[float, float]:
81     """ 以同窗宽归一化的 FDCT 为参考, 计算全图 PSNR 与 SSIM。"""
82     psnr_value = peak_signal_noise_ratio(reference, candidate, data_range=255)
83     ssim_value = structural_similarity(reference, candidate, data_range=255)
84     return float(psnr_value), float(ssim_value)
85
86
87 def adaptive_median_filter(image: np.ndarray, max_window: int = 7) -> np.ndarray:
88     """ 真正的自适应中值滤波: 窗口从 3x3 逐级增大到 max_window。"""
89     if max_window < 3 or max_window % 2 == 0:
90         raise ValueError("max_window 必须是大于等于 3 的奇数")
91
92     source = image.astype(np.uint8)
93     output = source.copy()
94     unresolved = np.ones(source.shape, dtype=bool)
95     last_median = source.copy()
96
97     for window in range(3, max_window + 1, 2):
98         kernel = np.ones((window, window), dtype=np.uint8)
99         local_min = cv2.erode(source, kernel, borderType=cv2.BORDER_REFLECT)
100        local_max = cv2.dilate(source, kernel, borderType=cv2.BORDER_REFLECT)
101        local_median = cv2.medianBlur(source, window)
102        last_median = local_median
103
104        stage_a = (local_median > local_min) & (local_median < local_max)
105        newly_resolved = unresolved & stage_a
106        stage_b = (source > local_min) & (source < local_max)
107
108        keep_original = newly_resolved & stage_b
109        use_median = newly_resolved & ~stage_b
110        output[keep_original] = source[keep_original]
111        output[use_median] = local_median[use_median]
112        unresolved &= ~stage_a
113
114    output[unresolved] = last_median[unresolved]
115    return output
116
117
118 def wavelet_denoise(
119     image: np.ndarray,
120     threshold: float,
121     wavelet: str = "db1",
122     level: int = 2,
123 ) -> np.ndarray:
124     """ 二维离散小波软阈值去噪。"""
125     coeffs = pywt.wavedec2(image.astype(np.float32), wavelet, level=level)
126     thresholded = [coeffs[0]]
127     for horizontal, vertical, diagonal in coeffs[1:]:
128         thresholded.append(
129             tuple(
130                 pywt.threshold(detail, threshold, mode="soft")
131                 for detail in (horizontal, vertical, diagonal)
132             )
133         )
134     reconstructed = pywt.waverec2(thresholded, wavelet)
135     reconstructed = reconstructed[: image.shape[0], : image.shape[1]]

```

```

136     return np.clip(np rint(reconstructed), 0, 255).astype(np.uint8)
137
138
139 def save_grayscale(path: Path, image: np.ndarray) -> None:
140     """ 保存纯 512×512 灰度图, 不添加标题、坐标轴或白边。 """
141     if not cv2.imwrite(str(path), image):
142         raise OSError(f" 图片保存失败: {path}")
143
144
145 def save_figure(fig: plt.Figure, basename: str) -> None:
146     fig.savefig(
147         FIGURES_DIR / f"{basename}.png",
148         dpi=300,
149         bbox_inches="tight",
150         facecolor="white",
151     )
152     fig.savefig(
153         FIGURES_DIR / f"{basename}.svg",
154         bbox_inches="tight",
155         facecolor="white",
156     )
157     plt.close(fig)
158
159
160 def parameter_text(parameters: dict[str, object]) -> str:
161     return ", ".join(f"{key}={value}" for key, value in parameters.items())
162
163
164 def main() -> None:
165     for directory in (RESULTS_DIR, FIGURES_DIR, IMAGES_DIR):
166         directory.mkdir(parents=True, exist_ok=True)
167
168     fdct_hu, fdct_dicom = read_dicom_hu(FDCT_PATH)
169     ldct_hu, ldct_dicom = read_dicom_hu(LDCT_PATH)
170
171     if fdct_hu.shape != ldct_hu.shape:
172         raise ValueError(f" 配对图像尺寸不一致: FDCT={fdct_hu.shape}, LDCT={ldct_hu.shape}")
173
174     pairing_tags = {
175         "Patient ID": "0010|0020",
176         "Image Position Patient": "0020|0032",
177         "Image Orientation Patient": "0020|0037",
178         "Pixel Spacing": "0028|0030",
179         "Slice Thickness": "0018|0050",
180         "Convolution Kernel": "0018|1210",
181         "Instance Number": "0020|0013",
182     }
183     pairing_metadata: dict[str, str] = {}
184     for label, tag in pairing_tags.items():
185         fdct_value = metadata(fdct_dicom, tag)
186         ldct_value = metadata(ldct_dicom, tag)
187         if fdct_value != ldct_value:
188             raise ValueError(
189                 f" 配对图像的 {label} 不一致: FDCT={fdct_value}, LDCT={ldct_value}"
190             )
191         pairing_metadata[label] = fdct_value
192     fdct_position = pairing_metadata["Image Position Patient"]
193
194     fdct = window_to_uint8(fdct_hu)
195     ldct = window_to_uint8(ldct_hu)
196
197     search_rows: list[dict[str, object]] = []
198     best: dict[str, dict[str, object]] = {}
199
200     def record(
201         method: str,
202         parameters: dict[str, object],
203         candidate: np.ndarray,

```

```

204     elapsed_seconds: float,
205 ) -> None:
206     psnr_value, ssim_value = evaluate(fdct, candidate)
207     row = {
208         "method": method,
209         "parameters": parameter_text(parameters),
210         "psnr_db": psnr_value,
211         "ssim": ssim_value,
212         "runtime_seconds": elapsed_seconds,
213     }
214     search_rows.append(row)
215     previous = best.get(method)
216     # 医学图像以结构保持为主: SSIM 优先, PSNR 作为同分决胜项。
217     if previous is None or (ssim_value, psnr_value) > (
218         float(previous["ssim"]),
219         float(previous["psnr_db"]),
220     ):
221         best[method] = {**row, "image": candidate.copy(), "parameter_dict": parameters}
222
223     for kernel_size in (3, 5, 7):
224         start = time.perf_counter()
225         candidate = cv2.blur(ldct, (kernel_size, kernel_size))
226         record("Mean", {"kernel": kernel_size}, candidate, time.perf_counter() - start)
227
228     for kernel_size, sigma in (
229         (3, 0.5),
230         (3, 1.0),
231         (5, 0.5),
232         (5, 1.0),
233         (5, 1.5),
234         (7, 1.0),
235         (7, 1.5),
236     ):
237         start = time.perf_counter()
238         candidate = cv2.GaussianBlur(
239             ldct,
240             (kernel_size, kernel_size),
241             sigmaX=sigma,
242             borderType=cv2.BORDER_REFLECT,
243         )
244         record(
245             "Gaussian",
246             {"kernel": kernel_size, "sigma": sigma},
247             candidate,
248             time.perf_counter() - start,
249         )
250
251     for max_window in (3, 5, 7):
252         start = time.perf_counter()
253         candidate = adaptive_median_filter(ldct, max_window=max_window)
254         record(
255             "AdaptiveMedian",
256             {"max_window": max_window},
257             candidate,
258             time.perf_counter() - start,
259         )
260
261     for h_value in (1, 2, 3, 4, 5, 7, 10, 15):
262         start = time.perf_counter()
263         candidate = cv2.fastNlMeansDenoising(
264             ldct,
265             None,
266             h=float(h_value),
267             templateWindowSize=7,
268             searchWindowSize=21,
269         )
270         record("NLM", {"h": h_value}, candidate, time.perf_counter() - start)
271

```

```

272     for threshold in (1, 2, 3, 4, 5, 8, 10, 15, 20):
273         start = time.perf_counter()
274         candidate = wavelet_denoise(ldct, threshold=float(threshold))
275         record(
276             "Wavelet",
277             {"wavelet": "db1", "level": 2, "threshold": threshold},
278             candidate,
279             time.perf_counter() - start,
280         )
281
282     baseline_psnr, baseline_ssim = evaluate(fdct, ldct)
283     method_order = ["Mean", "Gaussian", "AdaptiveMedian", "NLM", "Wavelet"]
284     final_rows: list[dict[str, object]] = [
285         {
286             "method": "LDCT",
287             "parameters": "none",
288             "psnr_db": baseline_psnr,
289             "ssim": baseline_ssim,
290             "delta_psnr_db": 0.0,
291             "delta_ssim": 0.0,
292         }
293     ]
294     for method in method_order:
295         chosen = best[method]
296         final_rows.append(
297             {
298                 "method": method,
299                 "parameters": chosen["parameters"],
300                 "psnr_db": chosen["psnr_db"],
301                 "ssim": chosen["ssim"],
302                 "delta_psnr_db": float(chosen["psnr_db"]) - baseline_psnr,
303                 "delta_ssim": float(chosen["ssim"]) - baseline_ssim,
304             }
305         )
306
307     best_overall = max(
308         (row for row in final_rows if row["method"] != "LDCT"),
309         key=lambda row: (float(row["ssim"]), float(row["psnr_db"])),
310     )
311
312     with (RESULTS_DIR / "parameter_search.csv").open("w", newline="", encoding="utf-8") as file:
313         writer = csv.DictWriter(file, fieldnames=list(search_rows[0].keys()))
314         writer.writeheader()
315         writer.writerows(search_rows)
316
317     with (RESULTS_DIR / "metrics.csv").open("w", newline="", encoding="utf-8") as file:
318         writer = csv.DictWriter(file, fieldnames=list(final_rows[0].keys()))
319         writer.writeheader()
320         writer.writerows(final_rows)
321
322     save_grayscale(IMAGES_DIR / "fdct_ground_truth.png", fdct)
323     save_grayscale(IMAGES_DIR / "ldct_input.png", ldct)
324     output_names = {
325         "Mean": "mean.png",
326         "Gaussian": "gaussian.png",
327         "AdaptiveMedian": "adaptive_median.png",
328         "NLM": "nlm.png",
329         "Wavelet": "wavelet.png",
330     }
331     for method, filename in output_names.items():
332         save_grayscale(IMAGES_DIR / filename, np.asarray(best[method]["image"]))
333
334     panels = [
335         ("FDCT ground truth", fdct),
336         ("LDCT input", ldct),
337         *[(method, np.asarray(best[method]["image"])) for method in method_order],
338     ]
339     fig, axes = plt.subplots(2, 4, figsize=(12, 6.4), constrained_layout=True)

```

```

340     for axis, (title, image) in zip(axes.flat, panels):
341         axis.imshow(image, cmap="gray", vmin=0, vmax=255, interpolation="nearest")
342         axis.set_title(title, fontsize=10)
343         axis.axis("off")
344     summary_axis = axes.flat[-1]
345     summary_axis.axis("off")
346     summary_axis.text(
347         0.06,
348         0.82,
349         "Selected result",
350         fontsize=11,
351         fontweight="bold",
352         va="top",
353         transform=summary_axis.transAxes,
354     )
355     summary_axis.text(
356         0.06,
357         0.66,
358         f"Method: {best_overall['method']}\n"
359         f"PSNR: {float(best_overall['psnr_db']):.3f} dB\n"
360         f"SSIM: {float(best_overall['ssim']):.5f}",
361         fontsize=9,
362         linespacing=1.45,
363         va="top",
364         transform=summary_axis.transAxes,
365     )
366     summary_axis.text(
367         0.06,
368         0.30,
369         "Selection rule:\nmaximize SSIM,\nthen PSNR",
370         fontsize=8,
371         color="#555555",
372         linespacing=1.4,
373         va="top",
374         transform=summary_axis.transAxes,
375     )
376     fig.suptitle("DICOM-based denoising comparison (lung window)", fontsize=14)
377     save_figure(fig, "denoising_comparison")
378
379     # 肺内血管区域局部放大，所有方法使用相同 ROI 与显示尺度。
380     y0, y1, x0, x1 = 150, 370, 45, 265
381     detail_panels = [
382         ("FDCT", fdct[y0:y1, x0:x1]),
383         ("LDCT", ldct[y0:y1, x0:x1]),
384         ("NLM", np.asarray(best["NLM"]["image"])[y0:y1, x0:x1]),
385         ("Wavelet", np.asarray(best["Wavelet"]["image"])[y0:y1, x0:x1]),
386     ]
387     fig, axes = plt.subplots(1, 4, figsize=(11, 3), constrained_layout=True)
388     for axis, (title, image) in zip(axes, detail_panels):
389         axis.imshow(image, cmap="gray", vmin=0, vmax=255, interpolation="nearest")
390         axis.set_title(title, fontsize=10)
391         axis.axis("off")
392     fig.suptitle("Same lung ROI at native pixel scale", fontsize=13)
393     save_figure(fig, "detail_comparison")
394
395     labels = [str(row["method"]) for row in final_rows]
396     psnr_values = [float(row["psnr_db"]) for row in final_rows]
397     ssim_values = [float(row["ssim"]) for row in final_rows]
398     colors = ["#7f7f7f", "#4c78a8", "#f58518", "#54a24b", "#e45756", "#72b7b2"]
399     fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), constrained_layout=True)
400     psnr_bars = axes[0].bar(labels, psnr_values, color=colors)
401     axes[0].set_ylabel("PSNR (dB)")
402     axes[0].set_ylim(0, max(psnr_values) * 1.18)
403     axes[0].set_title("Pixel fidelity")
404     axes[0].bar_label(psnr_bars, fmt="%.2f", fontsize=8)
405     axes[0].tick_params(axis="x", rotation=25)
406     ssim_bars = axes[1].bar(labels, ssim_values, color=colors)
407     axes[1].set_ylabel("SSIM")

```



```

408 axes[1].set_ylim(0, 1.08)
409 axes[1].set_title("Structural similarity")
410 axes[1].bar_label(ssim_bars, fmt="%.4f", fontsize=8)
411 axes[1].tick_params(axis="x", rotation=25)
412 fig.suptitle("Denoising metrics against paired FDCT", fontsize=14)
413 save_figure(fig, "metrics_comparison")
414
415 summary_lines = [
416     " 任务二：基于原始配对 DICOM 的传统滤波去噪",
417     "-" * 60,
418     f"FDCT: {FDCT_PATH}",
419     f"LDCT: {LDCT_PATH}",
420     f" 图像位置: {fdct_position}",
421     f" 图像尺寸: {fdct.shape}",
422     f" 像素间距: {pairing_metadata['Pixel Spacing']} mm",
423     f" 层厚: {pairing_metadata['Slice Thickness']} mm",
424     f" 重建核: {pairing_metadata['Convolution Kernel']}",
425     f" 肺窗: center={WINDOW_CENTER:g} HU, width={WINDOW_WIDTH:g} HU, "
426     f"range=[{WINDOW_MIN:g}, {WINDOW_MAX:g}] HU",
427     " 预处理: HU 截断后统一线性映射为 uint8 0~255",
428     " 参数选择: SSIM 优先, PSNR 作为同分决胜项",
429     " 评价范围: 完整 512x512 图像 (无标题、坐标轴和白边)",
430     "",
431     " 最终指标",
432     "-" * 60,
433     "Method          PSNR(dB)      SSIM          Delta PSNR    Delta SSIM",
434 ]
435 for row in final_rows:
436     summary_lines.append(
437         f"{str(row['method']):17s} "
438         f"{float(row['psnr_db']):9.3f} "
439         f"{float(row['ssim']):8.5f} "
440         f"{float(row['delta_psnr_db']):10.3f} "
441         f"{float(row['delta_ssim']):10.5f}"
442     )
443 summary_lines.extend(
444     [
445         "",
446         " 最优参数",
447         "-" * 60,
448         *[f"{method}: {best[method]['parameters']}" for method in method_order],
449         "",
450         f" 按 SSIM 优先规则, 综合最优方法: {best_overall['method']}",
451         f"PSNR={float(best_overall['psnr_db']):.3f} dB, "
452         f"SSIM={float(best_overall['ssim']):.5f}",
453         "",
454         " 边界说明: 本次只验证 L067 的一张严格配对切片; 不能据此推出所有患者均相同。",
455     ]
456 )
457 (RESULTS_DIR / "summary.txt").write_text("\n".join(summary_lines) + "\n", encoding="utf-8")
458
459 manifest = {
460     "command": f"{sys.executable} {Path(__file__).name}",
461     "python": sys.version,
462     "platform": platform.platform(),
463     "inputs": {
464         "fdct": {"path": str(FDCT_PATH), "sha256": sha256(FDCT_PATH)},
465         "ldct": {"path": str(LDCT_PATH), "sha256": sha256(LDCT_PATH)},
466     },
467     "window": {"center_hu": WINDOW_CENTER, "width_hu": WINDOW_WIDTH},
468     "pairing_checks": pairing_metadata,
469     "packages": {
470         "numpy": np.__version__,
471         "opencv": cv2.__version__,
472         "SimpleITK": sitk.Version_VersionString(),
473         "matplotlib": matplotlib.__version__,
474         "PyWavelets": pywt.__version__,
475         "scikit-image": skimage.__version__,

```

```
476         },
477         "selection_rule": "maximize SSIM; break ties with PSNR",
478         "best_parameters": {
479             method: best[method]["parameter_dict"] for method in method_order
480         },
481     }
482     (RESULTS_DIR / "reproducibility.json").write_text(
483         json.dumps(manifest, ensure_ascii=False, indent=2) + "\n",
484         encoding="utf-8",
485     )
486
487     print("\n".join(summary_lines))
488     print(f"\n 结果目录: {TASK_DIR}")
489
490
491 if __name__ == "__main__":
492     main()
```

任务三: task3/main.py

```
1 import cv2
2 import numpy as np
3 import SimpleITK as sitk
4 import matplotlib.pyplot as plt
5 from pathlib import Path
6
7 nlm_image=cv2.imread(str("nlm.png"), cv2.IMREAD_GRAYSCALE)
8 if nlm_image is None:
9     print("Failed to read the image.")
10 print(nlm_image.shape)
11 # 读取 NLM 后的图像
12
13 low_resolution_image=cv2.resize(nlm_image,(128,128), interpolation=cv2.INTER_AREA)
14 # 将图像缩小为 128*128
15
16 # 最邻近插值放大图像
17 nearest_image=cv2.resize(low_resolution_image,(512,512), interpolation=cv2.INTER_NEAREST)
18 cv2.imwrite("/home/sanxi/DIP/experiment1/task3/output/nearest_image.png", nearest_image)
19
20 # 双线性插值放大图像
21 bilinear_image=cv2.resize(low_resolution_image,(512,512), interpolation=cv2.INTER_LINEAR)
22 cv2.imwrite("/home/sanxi/DIP/experiment1/task3/output/bilinear_image.png", bilinear_image)
23
24 # 双三次插值放大图像
25 bicubic_image=cv2.resize(low_resolution_image,(512,512), interpolation=cv2.INTER_CUBIC)
26 cv2.imwrite("/home/sanxi/DIP/experiment1/task3/output/bicubic_image.png", bicubic_image)
```

任务四: task4/main.py

```

1  #!/usr/bin/env python3
2  """ 任务 4: 以配对 FDCT 为 Ground Truth 评价去噪与 4 倍插值重建。"""
3
4  from __future__ import annotations
5
6  import csv
7  import sys
8  from pathlib import Path
9  from typing import Any
10
11 import matplotlib
12
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16 from PIL import Image
17 from skimage.metrics import peak_signal_noise_ratio, structural_similarity
18
19
20 # ----- 中文支持 -----
21 plt.rcParams["font.sans-serif"] = [
22     "Noto Serif CJK SC",
23     "Noto Serif CJK JP",
24     "SimSun",
25     "DejaVu Sans",
26 ]
27 plt.rcParams["axes.unicode_minus"] = False
28
29
30 TASK_DIR = Path(__file__).resolve().parent
31 PROJECT_DIR = TASK_DIR.parent
32 RESULTS_DIR = TASK_DIR / "results"
33
34 WINDOW_MIN = -1350.0
35 WINDOW_MAX = 150.0
36
37 # 可选 ROI: 左上角坐标及宽高。没有可靠标注时保持 None。
38 ROI_X: int | None = None
39 ROI_Y: int | None = None
40 ROI_W: int | None = None
41 ROI_H: int | None = None
42
43 FDCT_FILENAME = "L067_FD_3_1.CT.0002.0001.2015.12.22.18.12.07.5968.358090401.IMA"
44 LDCT_FILENAME = "L067_QD_3_1.CT.0004.0001.2015.12.22.18.12.56.428910.358293547.IMA"
45
46 TASK2_IMAGE_PATHS = {
47     "FDCT Ground Truth": PROJECT_DIR / "task2" / "images" / "fdct_ground_truth.png",
48     "Original LDCT": PROJECT_DIR / "task2" / "images" / "ldct_input.png",
49     "Mean": PROJECT_DIR / "task2" / "images" / "mean.png",
50     "Gaussian": PROJECT_DIR / "task2" / "images" / "gaussian.png",
51     "Adaptive Median": PROJECT_DIR / "task2" / "images" / "adaptive_median.png",
52     "NLM": PROJECT_DIR / "task2" / "images" / "nlm.png",
53     "Wavelet": PROJECT_DIR / "task2" / "images" / "wavelet.png",
54 }
55
56 TASK3_IMAGE_PATHS = {
57     "NLM + Nearest": PROJECT_DIR / "task3" / "output" / "nearest_image.png",
58     "NLM + Bilinear": PROJECT_DIR / "task3" / "output" / "bilinear_image.png",
59     "NLM + Bicubic": PROJECT_DIR / "task3" / "output" / "bicubic_image.png",
60 }
61
62
63 def read_gray(path: Path) -> np.ndarray:
64     """ 用 Pillow 稳健读取含中文路径的单通道 uint8 灰度图。"""
65     if not path.is_file():
66         raise FileNotFoundError(f" 缺少输入文件: {path}")
67     try:

```

```

68     with Image.open(path) as image:
69         if image.mode not in {"L", "I;16", "I", "F"}:
70             raise ValueError(f" 要求单通道灰度图, 实际 mode={image.mode}: {path}")
71         array = np.asarray(image)
72     except (OSError, ValueError) as exc:
73         raise ValueError(f" 无法读取有效灰度图: {path}; 原因: {exc}") from exc
74     if array.ndim != 2:
75         raise ValueError(f" 要求二维单通道图, 实际 shape={array.shape}: {path}")
76     if array.dtype != np.uint8:
77         raise ValueError(f" 要求 uint8 图像, 实际 dtype={array.dtype}: {path}")
78     return np.ascontiguousarray(array)
79
80
81 def normalize_to_uint8(image_hu: np.ndarray) -> np.ndarray:
82     """ 按固定肺窗截断、归一化, 并转换为 uint8。"""
83     clipped = np.clip(np.asarray(image_hu, dtype=np.float32), WINDOW_MIN, WINDOW_MAX)
84     normalized = (clipped - WINDOW_MIN) / (WINDOW_MAX - WINDOW_MIN)
85     return np rint(normalized * 255.0).astype(np.uint8)
86
87
88 def read_dicom_and_preprocess(path: Path) -> tuple[np.ndarray, Any]:
89     """ 使用 SimpleITK 读取单张 DICOM/IMA 并完成统一肺窗预处理。"""
90     if not path.is_file():
91         raise FileNotFoundError(f" 找不到 DICOM/IMA: {path}")
92     try:
93         import SimpleITK as sitk
94     except ImportError as exc:
95         raise RuntimeError("DICOM 回退或逐像素核验需要安装 SimpleITK。") from exc
96     dicom = sitk.ReadImage(str(path))
97     array = np.squeeze(sitk.GetArrayFromImage(dicom))
98     if array.ndim != 2:
99         raise ValueError(f"DICOM 应为二维切片, 实际 shape={array.shape}: {path}")
100     return normalize_to_uint8(array), dicom
101
102
103 def find_unique_file(filename: str) -> Path:
104     """ 按 task2 记录的精确文件名, 在真实目录中定位唯一 DICOM。"""
105     matches = [path for path in PROJECT_DIR.rglob(filename) if path.is_file()]
106     if len(matches) != 1:
107         raise FileNotFoundError(
108             f" 无法唯一定位 {filename}, 实际匹配 {len(matches)} 个: {matches}"
109         )
110     return matches[0]
111
112
113 def validate_image(name: str, image: np.ndarray, path: Path) -> str:
114     """ 检查并记录图像的路径、shape、dtype 和灰度范围。"""
115     if image.ndim != 2:
116         raise ValueError(f"{name} 不是二维灰度图: shape={image.shape}, 路径={path}")
117     if image.dtype != np.uint8:
118         raise ValueError(f"{name} 不是 uint8: dtype={image.dtype}, 路径={path}")
119     line = (
120         f"{name}: path={path}; shape={image.shape}; dtype={image.dtype}; "
121         f"min={int(image.min())}; max={int(image.max())}"
122     )
123     print(line)
124     return line
125
126
127 def calculate_metrics(reference: np.ndarray, candidate: np.ndarray) -> tuple[float, float]:
128     """ 以 GT 为第一个参数计算 PSNR 和 SSIM。"""
129     if reference.shape != candidate.shape:
130         raise ValueError(
131             " 不允许对尺寸不同的图像直接计算指标: "
132             f"GT={reference.shape}, processed={candidate.shape}"
133         )
134     psnr = peak_signal_noise_ratio(reference, candidate, data_range=255)
135     ssim = structural_similarity(reference, candidate, data_range=255)

```

```

136     return float(psnr), float(ssim)
137
138
139 def verify_pairing(fdct_dicom: Any, ldct_dicom: Any) -> list[str]:
140     """ 核对两张 DICOM 是否为同一位置的配对切片。"""
141     tags = {
142         "Patient ID": "0010|0020",
143         "Image Position Patient": "0020|0032",
144         "Image Orientation Patient": "0020|0037",
145         "Pixel Spacing": "0028|0030",
146         "Slice Thickness": "0018|0050",
147         "Convolution Kernel": "0018|1210",
148         "Instance Number": "0020|0013",
149     }
150     lines: list[str] = []
151     for label, tag in tags.items():
152         fd_value = fdct_dicom.GetMetaData(tag).strip() if fdct_dicom.HasMetaDataKey(tag) else "N/A"
153         ld_value = ldct_dicom.GetMetaData(tag).strip() if ldct_dicom.HasMetaDataKey(tag) else "N/A"
154         if fd_value != ld_value:
155             raise ValueError(f" 配对核验失败: {label}: FDCT={fd_value}, LDCT={ld_value}")
156         lines.append(f"{label}: {fd_value}")
157     return lines
158
159
160 def load_reference_and_ldct() -> tuple[dict[str, np.ndarray], dict[str, Path], list[str]]:
161     """ 优先使用规范 task2 PNG; 不合格时回退读取原始 DICOM。"""
162     paths = {
163         "FDCT Ground Truth": TASK2_IMAGE_PATHS["FDCT Ground Truth"],
164         "Original LDCT": TASK2_IMAGE_PATHS["Original LDCT"],
165     }
166     try:
167         images = {name: read_gray(path) for name, path in paths.items()}
168         if images["FDCT Ground Truth"].shape != images["Original LDCT"].shape:
169             raise ValueError("task2 的 FDCT/LDCT PNG 尺寸不一致")
170         if images["FDCT Ground Truth"].shape != (512, 512):
171             raise ValueError(f"task2 PNG 不是预期 512x512: {images['FDCT Ground Truth'].shape}")
172
173         fdct_path = find_unique_file(FDCT_FILENAME)
174         ldct_path = find_unique_file(LDCT_FILENAME)
175         try:
176             fdct_check, fdct_dicom = read_dicom_and_preprocess(fdct_path)
177             ldct_check, ldct_dicom = read_dicom_and_preprocess(ldct_path)
178             if not np.array_equal(images["FDCT Ground Truth"], fdct_check):
179                 raise ValueError("task2 FDCT PNG 与统一肺窗 DICOM 像素不一致")
180             if not np.array_equal(images["Original LDCT"], ldct_check):
181                 raise ValueError("task2 LDCT PNG 与统一肺窗 DICOM 像素不一致")
182             pairing_lines = verify_pairing(fdct_dicom, ldct_dicom)
183             pairing_lines.insert(0, "PNG 与 DICOM 统一预处理结果逐像素一致。")
184         except RuntimeError:
185             pairing_lines = [
186                 " 当前解释器未安装 SimpleITK; 运行时使用通过规格检查的 task2 纯灰度 PNG。",
187                 "DICOM 配对依据 task2 的 reproducibility.json 与 summary.txt。",
188             ]
189         paths["FDCT DICOM"] = fdct_path
190         paths["LDCT DICOM"] = ldct_path
191         return images, paths, pairing_lines
192     except (FileNotFoundError, ValueError) as png_error:
193         print(f" 规范 PNG 不可用, 回退到原始 DICOM: {png_error}")
194         fdct_path = find_unique_file(FDCT_FILENAME)
195         ldct_path = find_unique_file(LDCT_FILENAME)
196         fdct, fdct_dicom = read_dicom_and_preprocess(fdct_path)
197         ldct, ldct_dicom = read_dicom_and_preprocess(ldct_path)
198         pairing_lines = verify_pairing(fdct_dicom, ldct_dicom)
199         return (
200             {"FDCT Ground Truth": fdct, "Original LDCT": ldct},
201             {"FDCT DICOM": fdct_path, "LDCT DICOM": ldct_path},

```

```

202         pairing_lines,
203     )
204
205
206 def rebuild_sr_from_nlm(nlm: np.ndarray) -> dict[str, np.ndarray]:
207     """ 当 task3 不是 512 输出时, 建立 128→512 公平评价分支。"""
208     try:
209         import cv2
210     except ImportError as exc:
211         raise RuntimeError(" 重建分支需要安装 opencv-python。") from exc
212     low_resolution = cv2.resize(nlm, (128, 128), interpolation=cv2.INTER_AREA)
213     return {
214         "NLM + Nearest": cv2.resize(low_resolution, (512, 512), interpolation=cv2.INTER_NEAREST),
215         "NLM + Bilinear": cv2.resize(low_resolution, (512, 512), interpolation=cv2.INTER_LINEAR),
216         "NLM + Bicubic": cv2.resize(low_resolution, (512, 512), interpolation=cv2.INTER_CUBIC),
217     }
218
219
220 def load_all_images() -> tuple[dict[str, np.ndarray], dict[str, Path], list[str], str]:
221     images, paths, pairing_lines = load_reference_and_ldct()
222     gt_shape = images["FDCT Ground Truth"].shape
223
224     missing = [str(path) for name, path in TASK2_IMAGE_PATHS.items() if name not in images and not
                path.is_file()]
225     if missing:
226         raise FileNotFoundError(" 缺少 task2 去噪结果: \n" + "\n".join(missing))
227     for name, path in TASK2_IMAGE_PATHS.items():
228         if name not in images:
229             images[name] = read_gray(path)
230             paths[name] = path
231         if images[name].shape != gt_shape:
232             raise ValueError(
233                 f"{name} 尺寸与 FDCT 不一致, 不静默 resize: "
234                 f"{images[name].shape} vs {gt_shape}, 路径={path}"
235             )
236
237     missing_sr = [str(path) for path in TASK3_IMAGE_PATHS.values() if not path.is_file()]
238     if missing_sr:
239         raise FileNotFoundError(" 缺少 task3 插值结果: \n" + "\n".join(missing_sr))
240     sr_images = {name: read_gray(path) for name, path in TASK3_IMAGE_PATHS.items()}
241     sr_shapes = {image.shape for image in sr_images.values()}
242     if sr_shapes == {gt_shape}:
243         sr_mode = "Task3 实际流程为 512→128→512; 直接使用三种 512×512 输出与 FDCT 比较。"
244         images.update(sr_images)
245         paths.update(TASK3_IMAGE_PATHS)
246     else:
247         sr_mode = (
248             f"Task3 输出尺寸为 {sorted(sr_shapes)}, 不能直接与 FDCT {gt_shape} 比较; "
249             " 已在 task4 内从 NLM 建立 INTER_AREA 128×128→ 三种插值 512×512 的评价分支。"
250         )
251         print(sr_mode)
252         images.update(rebuild_sr_from_nlm(images["NLM"]))
253         for name in TASK3_IMAGE_PATHS:
254             paths[name] = Path("task4 内存重建 (未覆盖 task3) ")
255     return images, paths, pairing_lines, sr_mode
256
257
258 def create_metrics_table(images: dict[str, np.ndarray]) -> list[dict[str, str]]:
259     gt = images["FDCT Ground Truth"]
260     specifications = [
261         ("Original", "LDCT", "Original LDCT"),
262         ("Denoising", "Mean", "Mean"),
263         ("Denoising", "Gaussian", "Gaussian"),
264         ("Denoising", "Adaptive Median", "Adaptive Median"),
265         ("Denoising", "NLM", "NLM"),
266         ("Denoising", "Wavelet", "Wavelet"),
267         ("Super Resolution", "NLM + Nearest", "NLM + Nearest"),
268         ("Super Resolution", "NLM + Bilinear", "NLM + Bilinear"),

```

```

269         ("Super Resolution", "NLM + Bicubic", "NLM + Bicubic"),
270     ]
271     rows: list[dict[str, str]] = []
272     for category, method, key in specifications:
273         psnr, ssim = calculate_metrics(gt, images[key])
274         rows.append(
275             {"Category": category, "Method": method, "PSNR_dB": f"{psnr:.4f}", "SSIM": f"{ssim:.5f}"}
276         )
277     rows.append({"Category": "Reference", "Method": "FDCT", "PSNR_dB": "GT", "SSIM": "GT"})
278     return rows
279
280
281 def metrics_lookup(rows: list[dict[str, str]]) -> dict[str, tuple[str, str]]:
282     return {
283         row["Method"]: (row["PSNR_dB"], row["SSIM"])
284         for row in rows
285         if row["Category"] != "Reference"
286     }
287
288
289 def panel_title(label: str, method: str, lookup: dict[str, tuple[str, str]]) -> str:
290     if method == "FDCT":
291         return f"{label}\n 参考真值"
292     psnr, ssim = lookup[method]
293     return f"{label}\nPSNR {psnr} dB | SSIM {ssim}"
294
295
296 def plot_panels(
297     panels: list[tuple[str, str, np.ndarray]],
298     rows: list[dict[str, str]],
299     output: Path,
300     columns: int,
301     figure_title: str,
302     empty_note: str | None = None,
303 ) -> None:
304     lookup = metrics_lookup(rows)
305     nrows = int(np.ceil(len(panels) / columns))
306     fig, axes = plt.subplots(
307         nrows,
308         columns,
309         figsize=(3.3 * columns, 3.5 * nrows),
310         constrained_layout=True,
311         squeeze=False,
312     )
313     for axis, (label, method, image) in zip(axes.flat, panels):
314         axis.imshow(image, cmap="gray", vmin=0, vmax=255, interpolation="nearest")
315         axis.set_title(panel_title(label, method, lookup), fontsize=7.5)
316         axis.axis("off")
317     unused_axes = list(axes.flat[len(panels) :])
318     if empty_note and unused_axes:
319         unused_axes[0].text(
320             0.5,
321             0.5,
322             empty_note,
323             ha="center",
324             va="center",
325             fontsize=7.5,
326             linespacing=1.5,
327             transform=unused_axes[0].transAxes,
328         )
329     unused_axes[0].axis("off")
330     unused_axes = unused_axes[1:]
331     for axis in unused_axes:
332         axis.axis("off")
333     fig.suptitle(figure_title, fontsize=10)
334     fig.savefig(output, dpi=300, bbox_inches="tight", facecolor="white")
335     fig.savefig(output.with_suffix(".pdf"), bbox_inches="tight", facecolor="white")

```



```

336 plt.close(fig)
337
338
339 def plot_denoising_comparison(images: dict[str, np.ndarray], rows: list[dict[str, str]]) -> None:
340     panels = [
341         ("原始 LDCT", "LDCT", images["Original LDCT"]),
342         ("均值滤波", "Mean", images["Mean"]),
343         ("高斯滤波", "Gaussian", images["Gaussian"]),
344         ("自适应中值滤波", "Adaptive Median", images["Adaptive Median"]),
345         ("NLM", "NLM", images["NLM"]),
346         ("小波软阈值", "Wavelet", images["Wavelet"]),
347         ("FDCT 真值", "FDCT", images["FDCT Ground Truth"]),
348     ]
349     best = max(
350         (row for row in rows if row["Category"] == "Denoising"),
351         key=lambda row: (float(row["SSIM"]), float(row["PSNR_dB"])),
352     )
353     note = (
354         "最优去噪\n"
355         f"{best['Method']}\n\n"
356         f"PSNR {best['PSNR_dB']} dB\n"
357         f"SSIM {best['SSIM']}\n\n"
358         "参考: 配对 FDCT"
359     )
360     plot_panels(
361         panels,
362         rows,
363         RESULTS_DIR / "comparison_denoising.png",
364         4,
365         "五种去噪方法与 FDCT 真值对比",
366         empty_note=note,
367     )
368
369
370 def plot_sr_comparison(images: dict[str, np.ndarray], rows: list[dict[str, str]]) -> None:
371     panels = [
372         ("NLM", "NLM", images["NLM"]),
373         ("NLM + 最近邻", "NLM + Nearest", images["NLM + Nearest"]),
374         ("NLM + 双线性", "NLM + Bilinear", images["NLM + Bilinear"]),
375         ("NLM + 双三次", "NLM + Bicubic", images["NLM + Bicubic"]),
376         ("FDCT 真值", "FDCT", images["FDCT Ground Truth"]),
377     ]
378     plot_panels(
379         panels,
380         rows,
381         RESULTS_DIR / "comparison_sr.png",
382         5,
383         "三种插值 4 倍重建与 FDCT 真值对比",
384     )
385
386
387 def plot_all_comparison(images: dict[str, np.ndarray], rows: list[dict[str, str]]) -> None:
388     panels = [
389         ("原始 LDCT", "LDCT", images["Original LDCT"]),
390         ("均值滤波", "Mean", images["Mean"]),
391         ("高斯滤波", "Gaussian", images["Gaussian"]),
392         ("自适应中值滤波", "Adaptive Median", images["Adaptive Median"]),
393         ("NLM", "NLM", images["NLM"]),
394         ("小波软阈值", "Wavelet", images["Wavelet"]),
395         ("NLM + 最近邻", "NLM + Nearest", images["NLM + Nearest"]),
396         ("NLM + 双线性", "NLM + Bilinear", images["NLM + Bilinear"]),
397         ("NLM + 双三次", "NLM + Bicubic", images["NLM + Bicubic"]),
398         ("FDCT 真值", "FDCT", images["FDCT Ground Truth"]),
399     ]
400     plot_panels(
401         panels,
402         rows,
403         RESULTS_DIR / "comparison_all.png",

```

```

404     5,
405     " 去噪与超分综合对比",
406 )
407
408
409 def plot_roi_comparison(images: dict[str, np.ndarray], rows: list[dict[str, str]]) -> bool:
410     values = (ROI_X, ROI_Y, ROI_W, ROI_H)
411     if all(value is None for value in values):
412         print("ROI 未设置: 不生成 comparison_roi.png。")
413         return False
414     if any(value is None for value in values):
415         raise ValueError("ROI_X、ROI_Y、ROI_W、ROI_H 必须全部设置或全部为 None")
416     x, y, width, height = (int(value) for value in values if value is not None)
417     if x < 0 or y < 0 or width <= 0 or height <= 0:
418         raise ValueError(f"ROI 参数无效: x={x}, y={y}, w={width}, h={height}")
419     image_height, image_width = images["FDCT Ground Truth"].shape
420     if x + width > image_width or y + height > image_height:
421         raise ValueError(f"ROI 超出图像范围: ROI=({x},{y},{width},{height}), image=({image_height,i
            mage_width})")
422
423     best_denoising = max(
424         (row for row in rows if row["Category"] == "Denoising"),
425         key=lambda row: (float(row["SSIM"]), float(row["PSNR_dB"])),
426     )["Method"]
427     keys = [
428         ("LDCT", "LDCT", "Original LDCT"),
429         (f" 最优: {best_denoising}", best_denoising, best_denoising),
430         (" 最近邻", "NLM + Nearest", "NLM + Nearest"),
431         (" 双线性", "NLM + Bilinear", "NLM + Bilinear"),
432         (" 双三次", "NLM + Bicubic", "NLM + Bicubic"),
433         ("FDCT 真值", "FDCT", "FDCT Ground Truth"),
434     ]
435     panels = [(label, method, images[key][y : y + height, x : x + width]) for label, method, key
        in keys]
436     plot_panels(panels, rows, RESULTS_DIR / "comparison_roi.png", 6, " 同一 ROI 局部结构对比")
437     return True
438
439
440 def write_metrics(rows: list[dict[str, str]]) -> str:
441     with (RESULTS_DIR / "metrics.csv").open("w", newline="", encoding="utf-8-sig") as file:
442         writer = csv.DictWriter(file, fieldnames=["Category", "Method", "PSNR_dB", "SSIM"])
443         writer.writeheader()
444         writer.writerows(rows)
445
446     header = f"{'Category':<18} {'Method':<20} {'PSNR (dB)':>12} {'SSIM':>10}"
447     lines = [
448         "Task4 metrics (reference: paired FDCT, data_range=255)",
449         "=" * len(header),
450         header,
451         "-" * len(header),
452     ]
453     for row in rows:
454         lines.append(
455             f"{row['Category']:<18} {row['Method']:<20} {row['PSNR_dB']:>12} {row['SSIM']:>10}"
456         )
457     table = "\n".join(lines)
458     (RESULTS_DIR / "metrics.txt").write_text(table + "\n", encoding="utf-8")
459     print("\n" + table)
460     return table
461
462
463 def main() -> None:
464     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
465     print(f" 项目目录: {PROJECT_DIR}")
466     print(" 开始检查输入; 不会修改 task1、task2、task3。")
467
468     images, paths, pairing_lines, sr_mode = load_all_images()
469     validation_lines = ["Task4 输入检查", "=" * 80]

```

```

470     for name, image in images.items():
471         validation_lines.append(validate_image(name, image, paths.get(name, Path(" 内存图像"))))
472     validation_lines.extend(["", "DICOM/PNG 配对核验", "-" * 80, *pairing_lines])
473     validation_lines.extend(["", "Task3 方案", "-" * 80, sr_model])
474     validation_lines.extend(
475         [
476             "",
477             " 原始 DICOM 路径",
478             "-" * 80,
479             f"FDCT: {paths['FDCT DICOM']}",
480             f"LDCT: {paths['LDCT DICOM']}",
481         ]
482     )
483     (RESULTS_DIR / "input_validation.txt").write_text(
484         "\n".join(validation_lines) + "\n", encoding="utf-8"
485     )
486
487     rows = create_metrics_table(images)
488     write_metrics(rows)
489     plot_denoising_comparison(images, rows)
490     plot_sr_comparison(images, rows)
491     plot_all_comparison(images, rows)
492     roi_created = plot_roi_comparison(images, rows)
493
494     expected = [
495         RESULTS_DIR / "metrics.csv",
496         RESULTS_DIR / "metrics.txt",
497         RESULTS_DIR / "input_validation.txt",
498         RESULTS_DIR / "comparison_denoising.png",
499         RESULTS_DIR / "comparison_sr.png",
500         RESULTS_DIR / "comparison_all.png",
501         RESULTS_DIR / "comparison_denoising.pdf",
502         RESULTS_DIR / "comparison_sr.pdf",
503         RESULTS_DIR / "comparison_all.pdf",
504     ]
505     if roi_created:
506         expected.append(RESULTS_DIR / "comparison_roi.png")
507         expected.append(RESULTS_DIR / "comparison_roi.pdf")
508     missing_outputs = [str(path) for path in expected if not path.is_file() or path.stat().st_size == 0]
509     if missing_outputs:
510         raise OSError(" 输出生成失败: \n" + "\n".join(missing_outputs))
511     print("\nTask4 完成。已生成: ")
512     for path in expected:
513         print(f"- {path} ({path.stat().st_size} bytes)")
514
515
516 if __name__ == "__main__":
517     try:
518         main()
519     except Exception as exc:
520         print(f"\nTask4 失败: {type(exc).__name__}: {exc}", file=sys.stderr)
521         raise

```

任务五: task5/task5-1.py

```

1  #-*- coding: utf-8 -*-
2  """
3  任务 5
4  """
5
6  import os
7  import cv2
8  import numpy as np
9  import pydicom
10 import matplotlib.pyplot as plt
11 from skimage.metrics import structural_similarity as ssim
12
13 LDCT_PATH = r"C:\Users\Sueye\Desktop\quarter_3mm\quarter_3mm\L067\quarter_3mm\L067_QD_3_1.CT.000_
    4.0001.2015.12.22.18.12.56.428910.358293547.IMA"
14
15 FDCT_PATH = r"C:\Users\Sueye\Desktop\full_3mm\full_3mm\L067\full_3mm\L067_FD_3_1.CT.0002.0001.20_
    15.12.22.18.12.07.5968.358090401.IMA"
16
17 OUT_DIR = 'results'
18 os.makedirs(OUT_DIR, exist_ok=True)
19
20 SCALE = 4    # 4 倍超分
21
22
23 # =====
24 # 1. 读取 .ima (DICOM) 并转 HU
25 # =====
26 def read_ima(path):
27     """ 读取 .ima / DICOM 文件, 返回 HU 值 2D 数组 """
28     ds = pydicom.dcmread(path, force=True)
29     arr = ds.pixel_array.astype(np.float32)
30
31     slope = float(getattr(ds, 'RescaleSlope', 1.0))
32     intercept = float(getattr(ds, 'RescaleIntercept', 0.0))
33     hu = arr * slope + intercept
34
35     if hu.ndim == 3:
36         hu = hu[0]    # 多帧时取第一帧
37     return hu
38
39
40 # =====
41 # 2. 肺窗 + 归一化到 0~255
42 # =====
43 def lung_window(hu, wl=-600, ww=1500):
44     """ 肺窗: 窗位 -600, 窗宽 1500, 归一化到 0~255 uint8 """
45     lo, hi = wl - ww / 2, wl + ww / 2
46     hu = np.clip(hu, lo, hi)
47     return ((hu - lo) / (hi - lo) * 255).astype(np.uint8)
48
49
50 # =====
51 # 3. 去噪方法 (只保留 NLM, 任务五顺序分析用)
52 # =====
53 def denoise_nlm(img, h=4):
54     # 与队友任务二最优参数保持一致: h=4
55     return cv2.fastNlMeansDenoising(img, None, h, 7, 21)
56
57
58 # =====
59 # 4. 4 倍插值 (双三次)
60 # =====
61 def up4(img, method='bicubic'):
62     interp = {
63         'nearest': cv2.INTER_NEAREST,
64         'bilinear': cv2.INTER_LINEAR,
65         'bicubic': cv2.INTER_CUBIC,

```

```

66     }[method]
67     return cv2.resize(img, None, fx=SCALE, fy=SCALE, interpolation=interp)
68
69
70 # =====
71 # 5. 评估指标
72 # =====
73 def psnr(a, b):
74     a = a.astype(np.float64)
75     b = b.astype(np.float64)
76     mse = np.mean((a - b) ** 2)
77     return 10 * np.log10(255.0 ** 2 / mse) if mse > 0 else float('inf')
78
79 def calc_ssim(a, b):
80     return ssim(a, b, data_range=255)
81
82
83 # =====
84 # 6. 主流程
85 # =====
86 def main():
87     # ---- 读取 ----
88     ld_hu = read_ima(LDCT_PATH)
89     fd_hu = read_ima(FDCT_PATH)
90
91     ld = lung_window(ld_hu)
92     fd = lung_window(fd_hu)
93
94     print('LDCT shape:', ld.shape, 'FDCT shape:', fd.shape)
95
96     # =====
97     # 6a. 顺序影响对比 (GT 为插值放大的 FDCT)
98     # =====
99     # 方案 A: 先降噪, 后超分
100    A = up4(denoise_nlm(ld, h=4), 'bicubic')
101    gtA = cv2.resize(fd, (A.shape[1], A.shape[0]),
102                    interpolation=cv2.INTER_CUBIC)
103    pA, sA = psnr(A, gtA), calc_ssim(A, gtA)
104
105    # 方案 B: 先超分, 后降噪
106    B_up = up4(ld, 'bicubic')
107    B = denoise_nlm(B_up, h=4)
108    gtB = cv2.resize(fd, (B.shape[1], B.shape[0]),
109                    interpolation=cv2.INTER_CUBIC)
110    pB, sB = psnr(B, gtB), calc_ssim(B, gtB)
111
112    print('\n=== 顺序影响对比 (GT 为插值放大的 FDCT) ===')
113    print(f'A 先降噪后超分: PSNR={pA:.2f} dB, SSIM={sA:.4f}')
114    print(f'B 先超分后降噪: PSNR={pB:.2f} dB, SSIM={sB:.4f}')
115
116    # =====
117    # 7. 可视化
118    # =====
119    plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
120    plt.rcParams['axes.unicode_minus'] = False
121
122    # 图 1: 顺序对比
123    fig, axes = plt.subplots(1, 4, figsize=(20, 5))
124    axes[0].imshow(ld, cmap='gray'); axes[0].set_title('LDCT 原图')
125    axes[1].imshow(A, cmap='gray'); axes[1].set_title('A 先降噪后超分')
126    axes[2].imshow(B, cmap='gray'); axes[2].set_title('B 先超分后降噪')
127    axes[3].imshow(gtA, cmap='gray'); axes[3].set_title('FDCT 真值')
128    for ax in axes:
129        ax.axis('off')
130    plt.tight_layout()
131    plt.savefig(os.path.join(OUT_DIR, 'fig_order.png'), dpi=150)
132
133    print(f'\n 图已保存到 {OUT_DIR}/')

```

```
134
135 # =====
136 # 8. 保存指标表
137 # =====
138 with open(os.path.join(OUT_DIR, 'metrics_task5.txt'), 'w',
139           encoding='utf-8') as f:
140     f.write('=== 顺序影响 (GT 为插值放大的 FDCT) ===\n')
141     f.write(f'A 先降噪后超分\tPSNR={pA:.2f}\tSSIM={sA:.4f}\n')
142     f.write(f'B 先超分后降噪\tPSNR={pB:.2f}\tSSIM={sB:.4f}\n')
143     f.write('\n 注：三种插值对比与各去噪 + 双三次汇总引用队友任务二、三、四结果，本脚本不重复。\n')
144
145 print('指标已保存到 results/metrics_task5.txt')
146
147
148 if __name__ == '__main__':
149     main()
```